

VERUNUM

End-to-End User, Dashboard & Device Flow

Application flow specification for the hardware / firmware developer

Dashboard Organisation administration	Backend REST + WebSocket	Device TAB5 + R503
Provision devices Create people Start enrollment View attendance	Organisation binding Authentication Commands Events & state	Connect Fingerprint capture Local match Attendance reporting

Purpose. This document defines how an organisation administrator uses Verunum and how the TAB5/R503 device interacts with the platform. It is the functional contract for the hardware developer.

Core rule: biometric templates remain on the device. The platform receives identity and attendance events, not raw fingerprint templates.

1. System at a glance

Organisation Admin	Verunum Backend	TAB5 / R503
1. Add device	2. Create pending provisioning	3. Device boots
4. Add person	5. Send enrollment command	6. Capture fingerprint
7. View result	8. Store status / event	9. Match fingerprint locally
10. View attendance	11. Record event	12. Send attendance event

2. Automatic device provisioning

The administrator does **not** manually enter an API key into the hardware. The device is automatically bound to the organisation when its MAC address matches a pending provisioning.

Dashboard: Devices → Add device. Enter the device name, location/details and MAC address. The MAC UI uses six two-character boxes:

AA | BB | CC | DD | EE | FF

The backend creates a temporary pending-provisioning record tied to the current organisation. The device may already be powered on.

Device → Backend: hello

```
{
  "type": "hello",
  "payload": {
    "mac_address": "AA:BB:CC:DD:EE:FF",
    "device_name": "Main Gate TAB5",
    "protocol_version": 1,
    "firmware_version": "1.0.0"
  }
}
```

The backend extracts the MAC address, searches pending provisioning records, and compares the MAC with the pending entry.

If the MAC matches:

The backend creates the permanent device record, binds the device to the organisation that created the pending provisioning, consumes the pending record, generates a unique API key, and returns it once to the device.

```
{
  "type": "hello.ack",
  "status": "provisioned",
  "payload": {
    "device_id": "dev_123",
    "organisation_id": "org_456",
    "api_key": "VERUNUM_DEVICE_KEY",
    "protocol_version": 1
  }
}
```

The device securely stores the API key. It is never hard-coded into the firmware. The API key is returned only for the successful first-time provisioning event.

3. Normal device reconnect

```
Device
|
| stored API key + WebSocket
v
Verunum
|
| authenticate + hello
v
hello.ack
|
v
ONLINE
```

On later boots or reconnects, the device uses its stored credential. It is not provisioned again and does not request a new key. Invalid/revoked credentials are rejected.

4. Attendant enrollment

Device provisioning and person enrollment are separate workflows. Provisioning binds a physical terminal to an organisation. Person enrollment creates the person's identity record and stores their fingerprint template locally on the selected terminal.

Dashboard: People → Add person. Create the person's profile. Then open the person's enrollment page and select an online TAB5.

Press **Click to start fingerprint intake**. The backend sends:

```
{
  "type": "enroll.start",
  "request_id": "req_001",
  "payload": {
    "user_id": "user_789",
    "finger_index": 1,
    "timeout_seconds": 30
  }
}
```

The TAB5 instructs the R503 to capture and verify the fingerprint. The template stays on the device and is associated with the supplied `user_id`.

```
{
  "type": "enroll.result",
  "request_id": "req_001",
  "payload": {
    "user_id": "user_789",
    "finger_index": 1,
    "status": "enrolled",
    "error": ""
  }
}
```

The backend validates the response and changes the dashboard status to Enrolled.

5. Attendance

Person	TAB5 / R503	Verunum
Places finger	Captures + matches locally	—
—	Creates event	Receives event
—	Queues if offline	Deduplicates + records
—	Receives ACK	Updates dashboard

Every attendance event must have a unique **event_id**. This prevents duplicate records when an offline event is retried.

```
{
  "type": "attendance",
  "event_id": "evt_20260920_000123",
  "payload": {
    "user_id": "user_789",
    "event": "clock_in",
    "timestamp": "2026-09-20T08:04:12Z",
    "method": "fingerprint"
  }
}
```

6. Offline and reconnect

```
ONLINE
↓ network lost
OFFLINE
↓ attendance occurs
LOCAL QUEUE
↓ network restored
RECONNECT + AUTHENTICATE
↓
SYNC event_id values
↓
SERVER ACK
↓
REMOVE acknowledged events
```

Use reconnect backoff such as 1s → 2s → 4s → 8s → 16s, capped around 30s. Preserve event_id during retries.

7. Device states

State	Meaning	Device behavior
Pending	Admin created provisioning; no permanent device yet.	Connect and send hello.
Provisioned	MAC matched and device was created.	Store API key securely.
Online	Authenticated WebSocket active.	Accept commands and report events.
Offline	No active connection.	Reconnect; retain local events.
Revoked	Credential/device disabled.	Stop normal operation.

8. Protocol rules

- hello is the first application message after connection.
- MAC + device name identify the physical terminal during initial provisioning.
- A matching pending MAC automatically binds the terminal to the pending organisation.
- The initial successful provisioning response contains the one-time API key.
- Future connections authenticate with the stored API key.
- Commands use request_id; attendance uses event_id.
- Raw fingerprint templates never leave the terminal.
- Attendance is queued locally when offline and synchronized after reconnect.
- Production uses WSS/TLS.

Device WebSocket endpoint: /api/v1/devices/ws

9. Hardware developer checklist

- Read MAC at runtime
- Connect to configured backend URL
- Send hello with MAC, name, protocol and firmware
- Handle first-time hello.ack and securely store API key
- Authenticate later connections
- Implement reconnect/backoff
- Implement enroll.start and R503 enrollment
- Return enroll.result with request_id
- Store fingerprint mapping locally
- Generate unique attendance event_id
- Queue attendance while offline
- Retry after reconnect and remove only after ACK
- Never transmit raw fingerprint templates

Implementation principle: the firmware should follow this contract exactly. Any new command or payload field should be agreed with the backend developer before implementation.